第12章 通过异常处理错误

Java的基本理念是“结构不佳的代码不能运行”。

发现错误的理想时机是在编译阶段，也就是在你试图运行程序之前。然而，编译期间并不能找出所有的错误，余下的问题必须在运行期间解决。这就需要错误源能通过某种方式，把适当的信息传递给某个接收者——该接收者将知道如何正确处理这个问题。

改进的错误恢复机制是提供代码健壮性的最强有力的方式。错误恢复在我们所编写的每一个程序中都是基本的要素，但是在Java中它显得格外重要，因为Java的主要目标之一就是创建供他人使用的程序构件。要想创建健壮的系统，它的每一个构件都必须是健壮的。Java使用异常来提供一致的错误报告模型，使得构件能够与客户端代码可靠地沟通问题。

Java中的异常处理的目的在于通过使用少于目前数量的代码来简化大型、可靠的程序的生成，并且通过这种方式可以使你更加自信：你的应用中没有未处理的错误。异常的相关知识学起来并非艰涩难懂，并且它属于那种可以使你的项目受益明显、立竿见影的特性之一。

因为异常处理是Java中唯一正式的错误报告机制，并且通过编译器强制执行，所以不学习异常处理的话，书中也就只能写出那么些例子了。本章将向读者介绍如何编写正确的异常处理程序，并将展示当方法出问题的时候，如何产生自定义的异常。

12.1 概念

C以及其他早期语言常常具有多种错误处理模式，这些模式往往建立在约定俗成的基础之上，而并不属于语言的一部分。通常会返回某个特殊值或者设置某个标志，并且假定接收者将对这个返回值或标志进行检查，以判定是否发生了错误。然而，随着时间的推移，人们发现，高傲的程序员们在使用程序库的时候更倾向于认为：“对，错误也许会发生，但那是别人造成的，不关我的事”。所以，程序员不去检查错误情形也就不足为奇了（何况对某些错误情形的检查确实很无聊[⊖]）。如果的确在每次调用方法的时候都彻底地进行错误检查，代码很可能会变得难以阅读。正是由于程序员还仍然用这些方式拼凑系统，所以他们拒绝承认这样一个事实：对于构造大型、健壮、可维护的程序而言，这种错误处理模式已经成为了主要障碍。

解决的办法是，用强制规定的形式来消除错误处理过程中随心所欲的因素。这种做法由来已久，对异常处理的实现可以追溯到20世纪60年代的操作系统，甚至于BASIC语言中的**on error goto**语句。而C++的异常处理机制基于Ada，Java中的异常处理则建立在C++的基础之上（尽管看上去更像Object Pascal）。

“异常”这个词有“我对此感到意外”的意思。问题出现了，你也许不清楚该如何处理，但你的确知道不应该置之不理；你要停下来，看看是不是有别人或在别的地方，能够处理这个问题。只是在当前的环境中还没有足够的信息来解决这个问题，所以就把这个问题提交到一个更高级别的环境中，在这里将作出正确的决定。

使用异常所带来的另一个相当明显的好处是，它往往能够降低错误处理代码的复杂度。如

⊖ 比如，C程序员检查**printf()**的返回值就是这样。

果不使用异常，那么就必须检查特定的错误，并在程序中的许多地方去处理它。而如果使用异常，那就不必在方法调用处进行检查，因为异常机制将保证能够捕获这个错误。并且，只需在一个地方处理错误，即所谓的*异常处理程序*中。这种方式不仅节省代码，而且把“描述在正常执行过程中做什么事”的代码和“出了问题怎么办”的代码相分离。总之，与以前的错误处理方法相比，异常机制使代码的阅读、编写和调试工作更加井井有条。 444

12.2 基本异常

异常情形（exceptional condition）是指阻止当前方法或作用域继续执行的问题。把异常情形与普通问题相区分很重要，所谓的普通问题是指，在当前环境下能得到足够的信息，总能处理这个错误。而对于异常情形，就不能继续下去了，因为*在当前环境下无法获得必要的信息来解决问题*。你所能做的就是从当前环境跳出，并且把问题提交给上一级环境。这就是抛出异常时所发生的事情。

除法就是一个简单的例子。除数有可能为0，所以先进行检查很有必要。但除数为0代表的究竟是什么意思呢？通过当前正在解决的问题环境，或许能知道该如何处理除数为0的情况。但如果这是一个意料之外的值，你也不清楚该如何处理，那就要抛出异常，而不是顺着原来的路径继续执行下去。

当抛出异常后，有几件事会随之发生。首先，同Java中其他对象的创建一样，将使用**new**在堆上创建异常对象。然后，当前的执行路径（它不能继续下去了）被终止，并且从当前环境中弹出对异常对象的引用。此时，异常处理机制接管程序，并开始寻找一个恰当的地方来继续执行程序。这个恰当的地方就是*异常处理程序*，它的任务是将程序从错误状态中恢复，以使程序能要么换一种方式运行，要么继续运行下去。

举一个抛出异常的简单例子。对于对象引用**t**，传给你的时候可能尚未被初始化。所以在使用这个对象引用调用其方法之前，会先对引用进行检查。可以创建一个代表错误信息的对象，并且将它从当前环境中“抛出”，这样就把错误信息传播到了“更大”的环境中。这被称为*抛出一个异常*，看起来像这样： 445

```
if(t == null)
  throw new NullPointerException();
```

这就抛出了异常，于是在当前环境下就不必再为这个问题操心了，它将在别的地方得到处理。具体是哪个“地方”后面很快就会介绍。

异常使得我们可以将每件事都当作一个事务来考虑，而异常可以看护着这些事务的底线“……事务的基本保障是我们所需的在分布式计算中的异常处理。事务是计算机中的合同法，如果出了什么问题，我们只需要放弃整个计算。”[⊖]我们还可以将异常看作是一种内建的恢复(undo)系统，因为（在细心使用的情况下）我们在程序中可以拥有各种不同的恢复点。如果程序的某部分失败了，异常将“恢复”到程序中某个已知的稳定点上。

异常最重要的方面之一就是如果发生问题，它们将不允许程序沿着其正常的路径继续走下去。在C和C++这样的语言中，这可真是个问题，尤其是C，它没有任何办法可以强制程序在出现问题时停止在某条路径上运行下去，因此我们有可能会较长时间地忽略了问题，从而陷入了完全不恰当的状态中。异常允许我们（如果没有其他手段）强制程序停止运行，并告诉我们出现了什么问题，或者（理想状态下）强制程序处理问题，并返回到稳定状态。

⊖ Jim Gray，www.acmqueue.org的一次访谈中提到，由于他的团队在事务方面的杰出贡献而成为图灵奖得主。

12.2.1 异常参数

与使用Java中的其他对象一样，我们总是用**new**在堆上创建异常对象，这也伴随着存储空间的分配和构造器的调用。所有标准异常类都有两个构造器：一个是默认构造器；另一个是接受字符串作为参数，以便能把相关信息放入异常对象的构造器：

```
throw new NullPointerException("t = null");
```

446 不久读者将看到，要把这个字符串的内容提取出来可以有多种不同的方法。

关键字**throw**将产生许多有趣的结果。在使用**new**创建了异常对象之后，此对象的引用将传给**throw**。尽管返回的异常对象其类型通常与方法设计的返回类型不同，但从效果上看，它就像是从方法“返回”的。可以简单地把异常处理看成一种不同的返回机制，当然若过分强调这种类比的话，就会有麻烦了。另外还能用抛出异常的方式从当前的作用域退出。在这两种情况下，将会返回一个异常对象，然后退出方法或作用域。

抛出异常与方法正常返回值的相似之处到此为止。因为异常返回的“地点”与普通方法调用返回的“地点”完全不同。(异常将在一个恰当的异常处理程序中得到解决，它的位置可能离异常被抛出的地方很远，也可能会跨越方法调用栈的许多层次。)

此外，能够抛出任意类型的**Throwable**对象，它是异常类型的根类。通常，对于不同类型的错误，要抛出相应的异常。错误信息可以保存在异常对象内部或者用异常类的名称来暗示。上一层环境通过这些信息来决定如何处理异常。(通常，异常对象中仅有的信息就是异常类型，除此之外不包含任何有意义的内容。)

12.3 捕获异常

要明白异常是如何被捕获的，必须首先理解*监控区域*（guarded region）的概念。它是一段可能产生异常的代码，并且后面跟着处理这些异常的代码。

12.3.1 try块

如果在方法内部抛出了异常（或者在方法内部调用的其他方法抛出了异常），这个方法将在抛出异常的过程中结束。要是不希望方法就此结束，可以在方法内设置一个特殊的块来捕获异常。因为在这个块里“尝试”各种（可能产生异常的）方法调用，所以称为try块。它是跟在**try**关键字之后的普通程序块：

```
try {
  // Code that might generate exceptions
}
```

447

对于不支持异常处理的程序语言，要想仔细检查错误，就得在每个方法调用的前后加上设置和错误检查的代码，甚至在每次调用同一方法时也得这么做。有了异常处理机制，可以把所有动作都放在try块里，然后只需在一个地方就可以捕获所有异常。这意味着代码将更容易编写和阅读，因为完成任务的代码没有与错误检查的代码混在一起。

12.3.2 异常处理程序

当然，抛出的异常必须在某处得到处理。这个“地点”就是*异常处理程序*，而且针对每个要捕获的异常，得准备相应的处理程序。异常处理程序紧跟在try块之后，以关键字**catch**表示：

```
try {
  // Code that might generate exceptions
} catch(Type1 id1) {
  // Handle exceptions of Type1
} catch(Type2 id2) {
```

```
    // Handle exceptions of Type2
  } catch(Type3 id3) {
    // Handle exceptions of Type3
  }

  // etc...
```

每个catch子句（异常处理程序）看起来就像是接收一个且仅接收一个特殊类型的参数的方法。可以在处理程序的内部使用标识符（**id1**，**id2**等等），这与方法参数的使用很相似。有时可能用不到标识符，因为异常的类型已经给了你足够的信息来对异常进行处理，但标识符并不可以省略。

异常处理程序必须紧跟在try块之后。当异常被抛出时，异常处理机制将负责搜寻参数与异常类型相匹配的第一个处理程序。然后进入catch子句执行，此时认为异常得到了处理。一旦catch子句结束，则处理程序的查找过程结束。注意，只有匹配的catch子句才能得到执行；这与**switch**语句不同，**switch**语句需要在每一个**case**后面跟一个**break**，以避免执行后续的**case**子句。 448

注意在try块的内部，许多不同的方法调用可能会产生类型相同的异常，而你只需要提供一个针对此类型的异常处理程序。

终止与恢复

异常处理理论上有两种基本模型。Java支持*终止模型*（它是Java和C++所支持的模型）[⊖]。在这种模型中，将假设错误非常关键，以至于程序无法返回到异常发生的地方继续执行。一旦异常被抛出，就表明错误已无法挽回，也不能回来继续执行。

另一种称为*恢复模型*。意思是异常处理程序的工作是修正错误，然后重新尝试调用出问题的方法，并认为第二次能成功。对于恢复模型，通常希望异常被处理之后能继续执行程序。如果想要用Java实现类似恢复的行为，那么在遇见错误时就不能抛出异常，而是调用方法来修正该错误。或者，把**try**块放在**while**循环里，这样就不断地进入**try**块，直到得到满意的结果。

长久以来，尽管程序员们使用的操作系统支持恢复模型的异常处理，但他们最终还是转向使用类似“终止模型”的代码，并且忽略恢复行为。所以虽然恢复模型开始显得很吸引人，但不是很实用。其中的主要原因可能是它所导致的耦合：恢复性的处理程序需要了解异常抛出的地点，这势必要包含依赖于抛出位置的非通用性代码。这增加了代码编写和维护的困难，对于异常可能会从许多地方抛出的大型程序来说，更是如此。

12.4 创建自定义异常

不必拘泥于Java中已有的异常类型。Java提供的异常体系不可能预见所有的希望加以报告的错误，所以可以自己定义异常类来表示程序中可能会遇到的特定问题。 449

要自己定义异常类，必须从已有的异常类继承，最好是选择意思相近的异常类继承（不过这样的异常并不容易找）。建立新的异常类型最简单的方法就是让编译器为你产生默认构造器，所以这几乎不用写多少代码：

```
//: exceptions/InheritingExceptions.java
// Creating your own exceptions.

class SimpleException extends Exception {}

public class InheritingExceptions {
  public void f() throws SimpleException {
```

⊖ 这与大多数语言的机制相同，包括C++、C#、Python和D等语言。

```
    System.out.println("Throw SimpleException from f()");
    throw new SimpleException();
  }
  public static void main(String[] args) {
    InheritingExceptions sed = new InheritingExceptions();
    try {
      sed.f();
    } catch(SimpleException e) {
      System.out.println("Caught it!");
    }
  }
} /* Output:
Throw SimpleException from f()
Caught it!
*///:~
```

编译器创建了默认构造器，它将自动调用基类的默认构造器。本例中不会得到像**SimpleException(String)** 这样的构造器，这种构造器也不实用。你将看到，对异常来说，最重要的部分就是类名，所以本例中建立的异常类在大多数情况下已经够用了。

本例的结果被打印到了控制台上，本书的输出显示系统正是在控制台上自动地捕获和测试这些结果的。但是，你也许想通过写入**System.err**而将错误发送给标准错误流。通常这比把错误信息输出到**System.out**要好，因为**System.out**也许会被重定向。如果把结果送到**System.err**，它就不会随**System.out**一起被重定向，这样更容易被用户注意。

450

也可以为异常类定义一个接受字符串参数的构造器：

```
//: exceptions/FullConstructors.java

class MyException extends Exception {
  public MyException() {}
  public MyException(String msg) { super(msg); }
}

public class FullConstructors {
  public static void f() throws MyException {
    System.out.println("Throwing MyException from f()");
    throw new MyException();
  }
  public static void g() throws MyException {
    System.out.println("Throwing MyException from g()");
    throw new MyException("Originated in g()");
  }
  public static void main(String[] args) {
    try {
      f();
    } catch(MyException e) {
      e.printStackTrace(System.out);
    }
    try {
      g();
    } catch(MyException e) {
      e.printStackTrace(System.out);
    }
  }
} /* Output:
Throwing MyException from f()
MyException
        at FullConstructors.f(FullConstructors.java:11)
        at FullConstructors.main(FullConstructors.java:19)
Throwing MyException from g()
MyException: Originated in g()
        at FullConstructors.g(FullConstructors.java:15)
        at FullConstructors.main(FullConstructors.java:24)
*///:~
```

451

新增的代码不长：两个构造器定义了**MyException**类型对象的创建方式。对于第二个构造器，使用**super**关键字明确调用了其基类构造器，它接受一个字符串作为参数。

在异常处理程序中，调用了在**Throwable**类声明（**Exception**即从此类继承）的**printStackTrace()**方法。就像从输出中看到的，它将打印“从方法调用处直到异常抛出处”的方法调用序列。这里，信息被发送到了**System.out**，并自动地被捕获和显示在输出中。但是，如果调用默认版本：

```
e.printStackTrace();
```

则信息将被输出到标准错误流。

练习1：(2) 编写一个类，在其**main()**方法的**try**块里抛出一个**Exception**类的对象。传递一个字符串参数给**Exception**的构造器。在**catch**子句里捕获此异常对象，并且打印字符串参数。添加一个**finally**子句，打印一条信息以证明这里确实得到了执行。

练习2：(1) 定义一个对象引用并初始化为**null**，尝试用此引用调用方法。把这个调用放在**try-catch**子句里以捕获异常。

练习3：(1) 编写能产生并能捕获**ArrayIndexOutOfBoundsException**异常的代码。

练习4：(2) 使用**extends**关键字建立一个自定义异常类。为这个类写一个接受字符串参数的构造器，把此参数保存在对象内部的字符串引用中。写一个方法显示此字符串。写一个**try-catch**子句，对这个新异常进行测试。

练习5：(3) 使用**while**循环建立类似“恢复模型”的异常处理行为，它将不断重复，直到异常不再抛出。

12.4.1 异常与记录日志

你可能还想使用**java.util.logging**工具将输出记录到日志中。尽管记录日志的全部细节是在http://MindView.net/Books/BetterJava的补充材料中介绍的，但是这里所使用的基本的日志记录功能还是相当简单易懂的：

452

```
//: exceptions/LoggingExceptions.java
// An exception that reports through a Logger.
import java.util.logging.*;
import java.io.*;

class LoggingException extends Exception {
  private static Logger logger =
    Logger.getLogger("LoggingException");
  public LoggingException() {
    StringWriter trace = new StringWriter();
    printStackTrace(new PrintWriter(trace));
    logger.severe(trace.toString());
  }
}

public class LoggingExceptions {
  public static void main(String[] args) {
    try {
      throw new LoggingException();
    } catch(LoggingException e) {
      System.err.println("Caught " + e);
    }
    try {
      throw new LoggingException();
    } catch(LoggingException e) {
      System.err.println("Caught " + e);
    }
  }
} /* Output: (85% match)
```

```
Aug 30, 2005 4:02:31 PM LoggingException <init>
SEVERE: LoggingException
        at
LoggingExceptions.main(LoggingExceptions.java:19)

Caught LoggingException
Aug 30, 2005 4:02:31 PM LoggingException <init>
SEVERE: LoggingException
        at
LoggingExceptions.main(LoggingExceptions.java:24)

Caught LoggingException
*///:~
```

静态的**Logger.getLogger()**方法创建了一个**String**参数相关联的**Logger**对象（通常与错误相关的包名和类名），这个**Logger**对象会将其输出发送到**System.err**。向**Logger**写入的最简单方式就是直接调用与日志记录消息的级别相关联的方法，这里使用的是**severe()**。为了产生日志记录消息，我们欲获取异常抛出处的栈轨迹，但是**printStackTrace()**不会默认地产生字符串。为了获取字符串，我们需要使用重载的**printStackTrace()**方法，它接受一个**java.io.PrintWriter**对象作为参数（这些都将在第18章中详细解释）。如果我们将一个**java.io.StringWriter**对象传递给这个**PrintWriter**的构造器，那么通过调用**toString()**方法，就可以将输出抽取为一个**String**。

453

尽管由于**LoggingException**将所有记录日志的基础设施都构建在异常自身中，使得它所使用的方式非常方便，并因此不需要客户端程序员的干预就可以自动运行，但是更常见的情形是我们需要捕获和记录其他人编写的异常，因此我们必须在异常处理程序中生成日志消息：

```
//: exceptions/LoggingException2.java
// Logging caught exceptions.
import java.util.logging.*;
import java.io.*;

public class LoggingExceptions2 {
  private static Logger logger =
    Logger.getLogger("LoggingExceptions2");
  static void logException(Exception e) {
    StringWriter trace = new StringWriter();
    e.printStackTrace(new PrintWriter(trace));
    logger.severe(trace.toString());
  }
  public static void main(String[] args) {
    try {
      throw new NullPointerException();
    } catch(NullPointerException e) {
      logException(e);
    }
  }
} /* Output: (90% match)
Aug 30, 2005 4:07:54 PM LoggingExceptions2 logException
SEVERE: java.lang.NullPointerException
        at
LoggingExceptions2.main(LoggingExceptions2.java:16)
*///:~
```

454

还可以更进一步自定义异常，比如加入额外的构造器和成员：

```
//: exceptions/ExtraFeatures.java
// Further embellishment of exception classes.
import static net.mindview.util.Print.*;

class MyException2 extends Exception {
  private int x;
  public MyException2() {}
  public MyException2(String msg) { super(msg); }
```

```
  public MyException2(String msg, int x) {
    super(msg);
    this.x = x;
  }
  public int val() { return x; }
  public String getMessage() {
    return "Detail Message: "+ x + " "+ super.getMessage();
  }
}

public class ExtraFeatures {
  public static void f() throws MyException2 {
    print("Throwing MyException2 from f()");
    throw new MyException2();
  }
  public static void g() throws MyException2 {
    print("Throwing MyException2 from g()");
    throw new MyException2("Originated in g()");
  }
  public static void h() throws MyException2 {
    print("Throwing MyException2 from h()");
    throw new MyException2("Originated in h()", 47);
  }
  public static void main(String[] args) {
    try {
      f();
    } catch(MyException2 e) {
      e.printStackTrace(System.out);
    }
    try {
      g();
    } catch(MyException2 e) {
      e.printStackTrace(System.out);
    }
    try {
      h();
    } catch(MyException2 e) {
      e.printStackTrace(System.out);
      System.out.println("e.val() = " + e.val());
    }
  }
} /* Output:
Throwing MyException2 from f()
MyException2: Detail Message: 0 null
        at ExtraFeatures.f(ExtraFeatures.java:22)
        at ExtraFeatures.main(ExtraFeatures.java:34)
Throwing MyException2 from g()
MyException2: Detail Message: 0 Originated in g()
        at ExtraFeatures.g(ExtraFeatures.java:26)
        at ExtraFeatures.main(ExtraFeatures.java:39)
Throwing MyException2 from h()
MyException2: Detail Message: 47 Originated in h()
        at ExtraFeatures.h(ExtraFeatures.java:30)
        at ExtraFeatures.main(ExtraFeatures.java:44)
e.val() = 47
*///:~
```

新的异常添加了字段**x**以及设定**x**值的构造器和读取数据的方法。此外，还覆盖了**Throwable.getMessage()**方法，以产生更详细的信息。对于异常类来说，**getMessage()**方法有点类似于**toString()**方法。

既然异常也是对象的一种，所以可以继续修改这个异常类，以得到更强的功能。但要记住，使用程序包的客户端程序员可能仅仅只是查看一下抛出的异常类型，其他的就不管了（大多数Java库里的异常都是这么用的），所以对异常所添加的其他功能也许根本用不上。

练习6：(1) 创建两个异常类，每一个都自动记录它们自己的日志，演示它们都可以正常运行。

456 **练习7**：(1) 修改练习3，使得**catch**子句可以将结果作为日志记录。

12.5 异常说明

Java鼓励人们把方法可能会抛出的异常告知使用此方法的客户端程序员。这是种优雅的做法，它使得调用者能确切知道写什么样的代码可以捕获所有潜在的异常。当然，如果提供了源代码，客户端程序员可以在源代码中查找**throw**语句来获知相关信息，然而程序库通常并不与源代码一起发布。为了预防这样的问题，Java提供了相应的语法（并强制使用这个语法），使你能以礼貌的方式告知客户端程序员某个方法可能会抛出的异常类型，然后客户端程序员就可以进行相应的处理。这就是*异常说明*，它属于方法声明的一部分，紧跟在形式参数列表之后。

异常说明使用了附加的关键字**throws**，后面接一个所有潜在异常类型的列表，所以方法定义可能看起来像这样：

但是，要是这样写：

```
void f() throws TooBig, TooSmall, DivZero { //...
```

就表示此方法不会抛出任何异常（除了从**RuntimeException**继承的异常，它们可以在没有异常

```
void f() { // ...
```

说明的情况下被抛出，这些将在后面进行讨论）。

代码必须与异常说明保持一致。如果方法里的代码产生了异常却没有进行处理，编译器会发现这个问题并提醒你：要么处理这个异常，要么就在异常说明中表明此方法将产生异常。通过这种自顶向下强制执行的异常说明机制，Java在编译时就可以保证一定水平的异常正确性。

不过还是有个能“作弊”的地方：可以声明方法将抛出异常，实际上却不抛出。编译器相信了这个声明，并强制此方法的用户像真的抛出异常那样使用这个方法。这样做的好处是，为异常先占个位子，以后就可以抛出这种异常而不用修改已有的代码。在定义抽象基类和接口时这种能力很重要，这样派生类或接口实现就能够抛出这些预先声明的异常。

这种在编译时被强制检查的异常称为*被检查的异常*。

练习8：(1) 定义一个类，令其方法抛出在练习2里定义的异常。不用异常说明，看看能否通过编译。然后加上异常说明，用**try-catch**子句测试该类和异常。 457

12.6 捕获所有异常

可以只写一个异常处理程序来捕获所有类型的异常。通过捕获异常类型的基类**Exception**，就可以做到这一点（事实上还有其他的基类，但**Exception**是同编程活动相关的基类）：

```
catch(Exception e) {
  System.out.println("Caught an exception");
}
```

这将捕获所有异常，所以最好把它放在处理程序列表的*末尾*，以防它抢在其他处理程序之前先把异常捕获了。

因为**Exception**是与编程有关的所有异常类的基类，所以它不会含有太多具体的信息，不过可以调用它从其基类**Throwable**继承的方法：

String getMessage()
String getLocalizedMessage()

用来获取详细信息，或用本地语言表示的详细信息。

String toString()

返回对**Throwable**的简单描述，要是有详细信息的话，也会把它包含在内。

void printStackTrace()
void printStackTrace(PrintStream)
void printStackTrace(java.io.PrintWriter)

打印**Throwable**和**Throwable**的调用栈轨迹。调用栈显示了“把你带到异常抛出地点”的方法调用序列。其中第一个版本输出到标准错误，后两个版本允许选择要输出的流（在第18章，你将学习这两种流的不同之处）。 458

Throwable fillInStackTrace()

用于在**Throwable**对象的内部记录栈帧的当前状态。这在程序重新抛出错误或异常（很快就会讲到）时很有用。

此外，也可以使用**Throwable**从其基类**Object**（也是所有类的基类）继承的方法。对于异常来说，**getClass()**也许是个很好用的方法，它将返回一个表示此对象类型的对象。然后可以使用**getName()**方法查询这个**Class**对象包含包信息的名称，或者使用只产生类名称的**getSimpleName()**方法。

下面的例子演示了如何使用**Exception**类型的方法：

```
//: exceptions/ExceptionMethods.java
// Demonstrating the Exception Methods.
import static net.mindview.util.Print.*;

public class ExceptionMethods {
  public static void main(String[] args) {
    try {
      throw new Exception("My Exception");
    } catch(Exception e) {
      print("Caught Exception");
      print("getMessage():" + e.getMessage());
      print("getLocalizedMessage():" +
        e.getLocalizedMessage());
      print("toString():" + e);
      print("printStackTrace():");
      e.printStackTrace(System.out);
    }
  }
} /* Output:
Caught Exception
getMessage():My Exception
getLocalizedMessage():My Exception
toString():java.lang.Exception: My Exception
printStackTrace():
java.lang.Exception: My Exception
        at ExceptionMethods.main(ExceptionMethods.java:8)
*///:~
```

459

可以发现每个方法都比前一个提供了更多的信息——实际上它们每一个都是前一个的超集。

练习9：(2) 定义三种新的异常类型。写一个类，在一个方法中抛出这三种异常。在**main()**里调用这个方法，仅用一个**catch**子句捕获这三种异常。

12.6.1 栈轨迹

printStackTrace()方法所提供的信息可以通过**getStackTrace()**方法来直接访问，这个方法将返回一个由栈轨迹中的元素所构成的数组，其中每一个元素都表示栈中的一桢。元素0是栈顶元素，并且是调用序列中的最后一个方法调用（这个**Throwable**被创建和抛出之处）。数组中的最后一个元素和栈底是调用序列中的第一个方法调用。下面的程序是一个简单的演示示例：

```
//: exceptions/WhoCalled.java
// Programmatic access to stack trace information.

public class WhoCalled {
  static void f() {
    // Generate an exception to fill in the stack trace
    try {
      throw new Exception();
    } catch (Exception e) {
      for(StackTraceElement ste : e.getStackTrace())
        System.out.println(ste.getMethodName());
    }
  }
  static void g() { f(); }
  static void h() { g(); }
  public static void main(String[] args) {
    f();
    System.out.println("--------------------------------");
    g();
    System.out.println("--------------------------------");
    h();
  }
} /* Output:
f
main
--------------------------------
f
g
main
--------------------------------
f
g
h
main
*///:~
```

460

这里，我们只打印了方法名，但实际上还可以打印整个**StackTraceElement**，它包含其他附件的信息。

12.6.2 重新抛出异常

有时希望把刚捕获的异常重新抛出，尤其是在使用**Exception**捕获所有异常的时候。既然已经得到了对当前异常对象的引用，可以直接把它重新抛出：

```
catch(Exception e) {
  System.out.println("An exception was thrown");
  throw e;
}
```

重抛异常会把异常抛给上一级环境中的异常处理程序，同一个**try**块的后续**catch**子句将被忽略。此外，异常对象的所有信息都得以保持，所以高一级环境中捕获此异常的处理程序可以从这个异常对象中得到所有信息。

如果只是把当前异常对象重新抛出，那么**printStackTrace()**方法显示的将是原来异常抛出点的调用栈信息，而并非重新抛出点的信息。要想更新这个信息，可以调用**fillInStackTrace()**方法，这将返回一个**Throwable**对象，它是通过把当前调用栈信息填入原来那个异常对象而建立的，就像这样：

```
//: exceptions/Rethrowing.java
// Demonstrating fillInStackTrace()

public class Rethrowing {
  public static void f() throws Exception {
    System.out.println("originating the exception in f()");
```

461

```
    throw new Exception("thrown from f()");
  }
  public static void g() throws Exception {
    try {
      f();
    } catch(Exception e) {
      System.out.println("Inside g(),e.printStackTrace()");
      e.printStackTrace(System.out);
      throw e;
    }
  }
  public static void h() throws Exception {
    try {
      f();
    } catch(Exception e) {
      System.out.println("Inside h(),e.printStackTrace()");
      e.printStackTrace(System.out);
      throw (Exception)e.fillInStackTrace();
    }
  }
  public static void main(String[] args) {
    try {
      g();
    } catch(Exception e) {
      System.out.println("main: printStackTrace()");
      e.printStackTrace(System.out);
    }
    try {
      h();
    } catch(Exception e) {
      System.out.println("main: printStackTrace()");
      e.printStackTrace(System.out);
    }
  }
} /* Output:
originating the exception in f()
Inside g(),e.printStackTrace()
java.lang.Exception: thrown from f()
        at Rethrowing.f(Rethrowing.java:7)
        at Rethrowing.g(Rethrowing.java:11)
        at Rethrowing.main(Rethrowing.java:29)
main: printStackTrace()
java.lang.Exception: thrown from f()
        at Rethrowing.f(Rethrowing.java:7)
        at Rethrowing.g(Rethrowing.java:11)
        at Rethrowing.main(Rethrowing.java:29)
originating the exception in f()
Inside h(),e.printStackTrace()
java.lang.Exception: thrown from f()
        at Rethrowing.f(Rethrowing.java:7)
        at Rethrowing.h(Rethrowing.java:20)
        at Rethrowing.main(Rethrowing.java:35)
main: printStackTrace()
java.lang.Exception: thrown from f()
        at Rethrowing.h(Rethrowing.java:24)
        at Rethrowing.main(Rethrowing.java:35)
*///:~
```

462

调用**fillInStackTrace()**的那一行就成了异常的新发生地了。

有可能在捕获异常之后抛出另一种异常。这么做的话，得到的效果类似于使用**fillInStackTrace()**，有关原来异常发生点的信息会丢失，剩下的是与新的抛出点有关的信息：

```
//: exceptions/RethrowNew.java
// Rethrow a different object from the one that was caught.

class OneException extends Exception {
```

```
  public OneException(String s) { super(s); }
}

class TwoException extends Exception {
  public TwoException(String s) { super(s); }
}

public class RethrowNew {
  public static void f() throws OneException {
    System.out.println("originating the exception in f()");
    throw new OneException("thrown from f()");
  }
  public static void main(String[] args) {
    try {
      try {
        f();
      } catch(OneException e) {
        System.out.println(
          "Caught in inner try, e.printStackTrace()");
        e.printStackTrace(System.out);
        throw new TwoException("from inner try");
      }
    } catch(TwoException e) {
      System.out.println(
        "Caught in outer try, e.printStackTrace()");
      e.printStackTrace(System.out);
    }
  }
} /* Output:
originating the exception in f()
Caught in inner try, e.printStackTrace()
OneException: thrown from f()
        at RethrowNew.f(RethrowNew.java:15)
        at RethrowNew.main(RethrowNew.java:20)
Caught in outer try, e.printStackTrace()
TwoException: from inner try
        at RethrowNew.main(RethrowNew.java:25)
*///:~
```

463

最后那个异常仅知道自己来自**main()**，而对**f()**一无所知。

永远不必为清理前一个异常对象而担心，或者说为异常对象的清理而担心。它们都是用**new**在堆上创建的对象，所以垃圾回收器会自动把它们清理掉。

12.6.3 异常链

常常会想要在捕获一个异常后抛出另一个异常，并且希望把原始异常的信息保存下来，这被称为*异常链*。在JDK 1.4以前，程序员必须自己编写代码来保存原始异常的信息。现在所有**Throwable**的子类在构造器中都可以接受一个*cause*（因由）对象作为参数。这个*cause*就用来表示原始异常，这样通过把原始异常传递给新的异常，使得即使在当前位置创建并抛出了新的异常，也能通过这个异常链追踪到异常最初发生的位置。

有趣的是，在**Throwable**的子类中，只有三种基本的异常类提供了带*cause*参数的构造器。它们是**Error**（用于Java虚拟机报告系统错误）、**Exception**以及**RuntimeException**。如果要把其他类型的异常链接起来，应该使用**initCause()**方法而不是构造器。

464

下面的例子能让你在运行时动态地向**DynamicFields**对象添加字段：

```
//: exceptions/DynamicFields.java
// A Class that dynamically adds fields to itself.
// Demonstrates exception chaining.
import static net.mindview.util.Print.*;

class DynamicFieldsException extends Exception {}
```

```
public class DynamicFields {
  private Object[][] fields;
  public DynamicFields(int initialSize) {
    fields = new Object[initialSize][2];
    for(int i = 0; i < initialSize; i++)
      fields[i] = new Object[] { null, null };
  }
  public String toString() {
    StringBuilder result = new StringBuilder();
    for(Object[] obj : fields) {
      result.append(obj[0]);
      result.append(": ");
      result.append(obj[1]);
      result.append("\n");
    }
    return result.toString();
  }
  private int hasField(String id) {
    for(int i = 0; i < fields.length; i++)
      if(id.equals(fields[i][0]))
        return i;
    return -1;
  }
  private int
  getFieldNumber(String id) throws NoSuchFieldException {
    int fieldNum = hasField(id);
    if(fieldNum == -1)
      throw new NoSuchFieldException();
    return fieldNum;
  }
  private int makeField(String id) {
    for(int i = 0; i < fields.length; i++)
      if(fields[i][0] == null) {
        fields[i][0] = id;
        return i;
      }
    // No empty fields. Add one:
    Object[][] tmp = new Object[fields.length + 1][2];
    for(int i = 0; i < fields.length; i++)
      tmp[i] = fields[i];
    for(int i = fields.length; i < tmp.length; i++)
      tmp[i] = new Object[] { null, null };
    fields = tmp;
    // Recursive call with expanded fields:
    return makeField(id);
  }
  public Object
  getField(String id) throws NoSuchFieldException {
    return fields[getFieldNumber(id)][1];
  }
  public Object setField(String id, Object value)
  throws DynamicFieldsException {
    if(value == null) {
      // Most exceptions don't have a "cause" constructor.
      // In these cases you must use initCause(),
      // available in all Throwable subclasses.
      DynamicFieldsException dfe =
        new DynamicFieldsException();
      dfe.initCause(new NullPointerException());
      throw dfe;
    }
    int fieldNumber = hasField(id);
    if(fieldNumber == -1)
      fieldNumber = makeField(id);
    Object result = null;
    try {
      result = getField(id); // Get old value
```

```
    } catch(NoSuchFieldException e) {
      // Use constructor that takes "cause":
      throw new RuntimeException(e);
    }
    fields[fieldNumber][1] = value;
    return result;
  }
  public static void main(String[] args) {
    DynamicFields df = new DynamicFields(3);
    print(df);
    try {
       df.setField("d", "A value for d");
       df.setField("number", 47);
       df.setField("number2", 48);
       print(df);
       df.setField("d", "A new value for d");
       df.setField("number3", 11);
       print("df: " + df);
       print("df.getField(\"d\") : " + df.getField("d"));
       Object field = df.setField("d", null); // Exception
     } catch(NoSuchFieldException e) {
       e.printStackTrace(System.out);
     } catch(DynamicFieldsException e) {
       e.printStackTrace(System.out);
     }
   }
} /* Output:
null: null
null: null
null: null

d: A value for d
number: 47
number2: 48

df: d: A new value for d
number: 47
number2: 48
number3: 11

df.getField("d") : A new value for d
DynamicFieldsException
        at DynamicFields.setField(DynamicFields.java:64)
        at DynamicFields.main(DynamicFields.java:94)
Caused by: java.lang.NullPointerException
        at DynamicFields.setField(DynamicFields.java:66)
        ... 1 more
*///:~
```

466

每个**DynamicFields**对象都含有一个数组，其元素是“成对的对象”。第一个对象表示字段标识符（一个字符串），第二个表示字段值，值的类型可以是除基本类型外的任意类型。当创建对象的时候，要合理估计一下需要多少字段。当调用**setField()**方法的时候，它将试图通过标识修改已有字段值，否则就建一个新的字段，并把值放入。如果空间不够了，将建立一个更长的数组，并把原来数组的元素复制进去。如果你试图为字段设置一个空值，将抛出一个**DynamicFieldsException**异常，它是通过使用**initCause()**方法把**NullPointerException**对象插入而建立的。

467

至于返回值，**setField()**将用**getField()**方法把此位置的旧值取出，这个操作可能会抛出**NoSuchFieldException**异常。如果客户端程序员调用了**getField()**方法，那么他就有责任处理这个可能抛出的 **NoSuchFieldException**异常，但如果异常是从**setField()**方法里抛出的，这种情况将被视为编程错误，所以就使用接受*cause*参数的构造器把**NoSuchFieldException**异常转换为

RuntimeException异常。

你会注意到，**toString()**方法使用了一个**StringBuilder**来创建其结果。在第13章中你将会了解到更多的关于**StringBuilder**的知识，但是只要你编写设计循环的**toString()**方法，通常都会想使用它，就像本例一样。

练习10：(2) 为一个类定义两个方法：**f()**和**g()**。在**g()**里，抛出一个自定义的新异常。在**f()**里，调用**g()**，捕获它抛出的异常，并且在**catch**子句里抛出另一个异常（自定义的第二种异常）。在**main()**里测试代码。

练习11：(1) 重复上一个练习，但是在**catch**子句里把**g()**要抛出的异常包装成一个**RuntimeException**。

12.7 Java标准异常

Throwable这个Java类被用来表示任何可以作为异常被抛出的类。**Throwable**对象可分为两种类型（指从**Throwable**继承而得到的类型）：**Error**用来表示编译时和系统错误（除特殊情况外，一般不用你关心）；**Exception**是可以被抛出的基本类型，在Java类库、用户方法以及运行时故障中都可能抛出**Exception**型异常。所以Java程序员关心的基类型通常是**Exception**。

要想对异常有全面的了解，最好去浏览一下HTML格式的Java文档（可以从java.sun.com下载）。为了对不同的异常有个感性的认识，这么做是值得的。但很快你就会发现，这些异常除了名称外其实都差不多。同时，Java中异常的数目在持续增加，所以在书中简单罗列它们毫无意义。所使用的第三方类库也可能会有自己的异常。对异常来说，关键是理解概念以及如何使用。 468

异常的基本的概念是用名称代表发生的问题，并且异常的名称应该可以望文知意。异常并非全是在**java.lang**包里定义的；有些异常是用来支持其他像**util**、**net**和**io**这样的程序包，这些异常可以通过它们的完整名称或者从它们的父类中看出端倪。比如，所有的输入/输出异常都是从**java.io.IOException**继承而来的。

12.7.1 特例：RuntimeException

在本章的第一个例子中：

```
if(t == null)
  throw new NullPointerException();
```

如果必须对传递给方法的每个引用都检查其是否为**null**（因为无法确定调用者是否传入了非法引用），这听起来着实吓人。幸运的是，这不必由你亲自来做，它属于Java的标准运行时检测的一部分。如果对**null**引用进行调用，Java会自动抛出**NullPointerException**异常，所以上述代码是多余的，尽管你也许想要执行其他的检查以确保**NullPointerException**不会出现。

属于运行时异常的类型有很多，它们会自动被Java虚拟机抛出，所以不必在异常说明中把它们列出来。这些异常都是从**RuntimeException**类继承而来，所以既体现了继承的优点，使用起来也很方便。这构成了一组具有相同特征和行为的异常类型。并且，也不再需要在异常说明中声明方法将抛出**RuntimeException**类型的异常（或者任何从**RuntimeException**继承的异常），它们也被称为“*不受检查异常*”。这种异常属于错误，将被自动捕获，就不用你亲自动手了。要是自己去检查**RuntimeException**的话，代码就显得太混乱了。不过尽管通常不用捕获**RuntimeException**异常，但还是可以在代码中抛出**RuntimeException**类型的异常。 469

如果不捕获这种类型的异常会发生什么事呢？因为编译器没有在这个问题上对异常说明进行强制检查，**RuntimeException**类型的异常也许会穿越所有的执行路径直达**main()**方法，而不会被捕获。要明白到底发生了什么，可以试试下面的例子：

```
//: exceptions/NeverCaught.java
// Ignoring RuntimeExceptions.
// {ThrowsException}

public class NeverCaught {
  static void f() {
    throw new RuntimeException("From f()");
  }
  static void g() {
    f();
  }
  public static void main(String[] args) {
    g();
  }
} ///:~
```

可能读者已经发现，**RuntimeException**（或任何从它继承的异常）是一个特例。对于这种异常类型，编译器不需要异常说明，其输出被报告给了**System.err**：

```
Exception in thread "main" java.lang.RuntimeException: From f()
        at NeverCaught.f(NeverCaught.java:7)
        at NeverCaught.g(NeverCaught.java:10)
        at NeverCaught.main(NeverCaught.java:13)
```

所以答案是：如果**RuntimeException**没有被捕获而直达**main()**，那么在程序退出前将调用异常的**printStackTrace()**方法。

请务必记住：只能在代码中忽略**RuntimeException**（及其子类）类型的异常，其他类型异常的处理都是由编译器强制实施的。究其原因，**RuntimeException**代表的是编程错误：

470

1) 无法预料的错误。比如从你控制范围之外传递进来的**null**引用。

2) 作为程序员，应该在代码中进行检查的错误。（比如对于**ArrayIndexOutOfBoundsException**，就得注意一下数组的大小了。）在一个地方发生的异常，常常会在另一个地方导致错误。

你会发现在这些情况下使用异常很有好处，它们能给调试带来便利。

值得注意的是：不应把Java的异常处理机制当成是单一用途的工具。是的，它被设计用来处理一些烦人的运行时错误，这些错误往往是由代码控制能力之外的因素导致的；然而，它对于发现某些编译器无法检测到的编程错误，也是非常重要的。

练习12：(3) 修改**innerclasses/Sequence.java**，使其在你试图向其中放置过多地元素时，抛出一个合适的异常。

12.8 使用finally进行清理

对于一些代码，可能会希望无论**try**块中的异常是否抛出，它们都能得到执行。这通常适用于内存回收之外的情况（因为回收由垃圾回收器完成）。为了达到这个效果，可以在异常处理程序后面加上**finally**子句[⊖]。完整的异常处理程序看起来像这样：

```
try {
  // The guarded region: Dangerous activities
  // that might throw A, B, or C
} catch(A a1) {
  // Handler for situation A
} catch(B b1) {
  // Handler for situation B
} catch(C c1) {
  // Handler for situation C
```

471

⊖ C++中的异常处理没有**finally**子句，它依赖析构函数来达到清理的目的。

```
} finally {
  // Activities that happen every time
}
```

为了证明**finally**子句总能运行，可以试试下面这个程序：

```
//: exceptions/FinallyWorks.java
// The finally clause is always executed.

class ThreeException extends Exception {}

public class FinallyWorks {
  static int count = 0;
  public static void main(String[] args) {
    while(true) {
      try {
        // Post-increment is zero first time:
        if(count++ == 0)
          throw new ThreeException();
        System.out.println("No exception");
      } catch(ThreeException e) {
        System.out.println("ThreeException");
      } finally {
        System.out.println("In finally clause");
        if(count == 2) break; // out of "while"
      }
    }
  }
} /* Output:
ThreeException
In finally clause
No exception
In finally clause
*///:~
```

可以从输出中发现，无论异常是否被抛出，**finally**子句总能被执行。

这个程序也给了我们一些思路，当Java中的异常不允许我们回到异常抛出的地点时，那么该如何应对呢？如果把**try**块放在循环里，就建立了一个“程序继续执行之前必须要达到”的条件。还可以加入一个**static**类型的计数器或者别的装置，使循环在放弃以前能尝试一定的次数。这将使程序的健壮性更上一个台阶。 472

12.8.1 finally用来做什么

对于没有垃圾回收和析构函数自动调用机制[⊖]的语言来说，**finally**非常重要。它能使程序员保证：无论**try**块里发生了什么，内存总能得到释放。但Java有垃圾回收机制，所以内存释放不再是问题。而且，Java也没有析构函数可供调用。那么，Java在什么情况下才能用到**finally**呢？

当要把除内存之外的资源恢复到它们的初始状态时，就要用到**finally**子句。这种需要清理的资源包括：已经打开的文件或网络连接，在屏幕上画的图形，甚至可以是外部世界的某个开关，如下面例子所示：

```
//: exceptions/Switch.java
import static net.mindview.util.Print.*;

public class Switch {
  private boolean state = false;
  public boolean read() { return state; }
  public void on() { state = true; print(this); }
  public void off() { state = false; print(this); }
```

⊖ 析构函数是“当对象不再被使用的时候”会被调用的函数。你总能确切地知道析构函数被调用的时间和地点。C++能自动调用析构函数，而C#（它更像Java）里面会有自动进行清理的机制。

```
  public String toString() { return state ? "on" : "off"; }
} ///:~

//: exceptions/OnOffException1.java
public class OnOffException1 extends Exception {} ///:~

//: exceptions/OnOffException2.java
public class OnOffException2 extends Exception {} ///:~

//: exceptions/OnOffSwitch.java
// Why use finally?

public class OnOffSwitch {
  private static Switch sw = new Switch();
  public static void f()
  throws OnOffException1,OnOffException2 {}
  public static void main(String[] args) {
    try {
      sw.on();
      // Code that can throw exceptions...
      f();
      sw.off();
    } catch(OnOffException1 e) {
      System.out.println("OnOffException1");
      sw.off();
    } catch(OnOffException2 e) {
      System.out.println("OnOffException2");
      sw.off();
    }
  }
} /* Output:
on
off
*///:~
```

473

程序的目的是要确保**main()**结束的时候开关必须是关闭的，所以在每个**try**块和异常处理程序的末尾都加入了对**sw.off()**方法的调用。但也可能有这种情况：异常被抛出，但没被处理程序捕获，这时**sw.off()**就得不到调用。但是有了**finally**，只要把**try**块中的清理代码移放在一处即可：

```
//: exceptions/WithFinally.java
// Finally Guarantees cleanup.

public class WithFinally {
  static Switch sw = new Switch();
  public static void main(String[] args) {
    try {
      sw.on();
      // Code that can throw exceptions...
      OnOffSwitch.f();
    } catch(OnOffException1 e) {
      System.out.println("OnOffException1");
    } catch(OnOffException2 e) {
      System.out.println("OnOffException2");
    } finally {
      sw.off();
    }
  }
} /* Output:
on
off
*///:~
```

474

这里**sw.off()**被移到一处，并且保证在任何情况下都能得到执行。

甚至在异常没有被当前的异常处理程序捕获的情况下，异常处理机制也会在跳到更高一层的异常处理程序之前，执行**finally**子句：

```
//: exceptions/AlwaysFinally.java
// Finally is always executed.
import static net.mindview.util.Print.*;

class FourException extends Exception {}

public class AlwaysFinally {
  public static void main(String[] args) {
    print("Entering first try block");
    try {
      print("Entering second try block");
      try {
        throw new FourException();
      } finally {
        print("finally in 2nd try block");
      }
    } catch(FourException e) {
      System.out.println(
        "Caught FourException in 1st try block");
    } finally {
      System.out.println("finally in 1st try block");
    }
  }
} /* Output:
Entering first try block
Entering second try block
finally in 2nd try block
Caught FourException in 1st try block
finally in 1st try block
*///:~
```

475

当涉及**break**和**continue**语句的时候，**finally**子句也会得到执行。请注意，如果把**finally**子句和带标签的**break**及**continue**配合使用，在Java里就没必要使用**goto**语句了。

练习13：(2) 修改练习9，加一个**finally**子句。验证一下，即便是抛出**NullPointerException**异常，**finally**子句也会得到执行。

练习14：(2) 试说明，在**OnOffSwitch.java**的**try**块内部抛出**RuntimeException**，程序可能会出现错误。

练习15：(2) 试说明，在**WithFinally.java**的**try**块内部抛出**RuntimeException**，程序不会出现错误。

12.8.2 在return中使用finally

因为**finally**子句总是会执行的，所以在一个方法中，可以从多个点返回，并且可以保证重要的清理工作仍旧会执行：

```
//: exceptions/MultipleReturns.java
import static net.mindview.util.Print.*;

public class MultipleReturns {
  public static void f(int i) {
    print("Initialization that requires cleanup");
    try {
      print("Point 1");
      if(i == 1) return;
      print("Point 2");
      if(i == 2) return;
      print("Point 3");
      if(i == 3) return;
      print("End");
      return;
    } finally {
      print("Performing cleanup");
    }
```

```
    }
    public static void main(String[] args) {
      for(int i = 1; i <= 4; i++)
        f(i);
    }
  } /* Output:
  Initialization that requires cleanup
  Point 1
  Performing cleanup
  Initialization that requires cleanup
  Point 1
  Point 2
  Performing cleanup
  Initialization that requires cleanup
  Point 1
  Point 2
  Point 3
  Performing cleanup
  Initialization that requires cleanup
  Point 1
  Point 2
  Point 3
  End
  Performing cleanup
  *///:~
```

476

从输出中可以看出，在**finally**类内部，从何处返回无关紧要。

练习16：(2) 修改**reusing/CADSystem.java**，以演示从**try-finally**的中间返回仍旧会执行正确的清理。

练习17：(3) 修改**polymorphism/Frog.java**，使其使用**try-finally**来保证正确的清理，并展示即使在**try-finally**的中间返回，它也可以起作用。

12.8.3 缺憾：异常丢失

遗憾的是，Java的异常实现也有瑕疵。异常作为程序出错的标志，决不应该被忽略，但它还是有可能被轻易地忽略。用某些特殊的方式使用**finally**子句，就会发生这种情况：

```
//: exceptions/LostMessage.java
// How an exception can be lost.

class VeryImportantException extends Exception {
  public String toString() {
    return "A very important exception!";
  }
}

class HoHumException extends Exception {
  public String toString() {
    return "A trivial exception";
  }
}

public class LostMessage {
  void f() throws VeryImportantException {
    throw new VeryImportantException();
  }
  void dispose() throws HoHumException {
    throw new HoHumException();
  }
  public static void main(String[] args) {
    try {
      LostMessage lm = new LostMessage();
      try {
        lm.f();
```

477

```
      } finally {
        lm.dispose();
      }
    } catch(Exception e) {
      System.out.println(e);
    }
  }
} /* Output:
A trivial exception
*///:~
```

从输出中可以看到，**VeryImportantException**不见了，它被**finally**子句里的 **HoHumException**所取代。这是相当严重的缺陷，因为异常可能会以一种比前面例子所示更微妙和难以察觉的方式完全丢失。相比之下，C++把“前一个异常还没处理就抛出下一个异常”的情形看成是糟糕的编程错误。也许在Java的未来版本中会修正这个问题（另一方面，要把所有抛出异常的方法，如上例中的**dispose()**方法，全部打包放到**try-catch**子句里面）。

一种更加简单的丢失异常的方式是从**finally**子句中返回：

478

```
//: exceptions/ExceptionSilencer.java

public class ExceptionSilencer {
  public static void main(String[] args) {
    try {
      throw new RuntimeException();
    } finally {
      // Using 'return' inside the finally block
      // will silence any thrown exception.
      return;
    }
  }
} ///:~
```

如果运行这个程序，就会看到即使抛出了异常，它也不会产生任何输出。

练习18：(3) 为**LostMessage.java**添加第二层异常丢失，以便用第三个异常来替代**HoHumException**异常。

练习19：(2) 通过确保**finally**子句中的调用，来修复**LostMessage.java**中的问题。

12.9 异常的限制

当覆盖方法的时候，只能抛出在基类方法的异常说明里列出的那些异常。这个限制很有用，因为这意味着，当基类使用的代码应用到其派生类对象的时候，一样能够工作（当然，这是面向对象的基本概念），异常也不例外。

下面例子演示了这种（在编译时）施加在异常上面的限制：

```
//: exceptions/StormyInning.java
// Overridden methods may throw only the exceptions
// specified in their base-class versions, or exceptions
// derived from the base-class exceptions.

class BaseballException extends Exception {}
class Foul extends BaseballException {}
class Strike extends BaseballException {}
abstract class Inning {
  public Inning() throws BaseballException {}
  public void event() throws BaseballException {
    // Doesn't actually have to throw anything
  }
  public abstract void atBat() throws Strike, Foul;
  public void walk() {} // Throws no checked exceptions
```

479

```
}

class StormException extends Exception {}
class RainedOut extends StormException {}
class PopFoul extends Foul {}

interface Storm {
  public void event() throws RainedOut;
  public void rainHard() throws RainedOut;
}

public class StormyInning extends Inning implements Storm {
  // OK to add new exceptions for constructors, but you
  // must deal with the base constructor exceptions:
  public StormyInning()
    throws RainedOut, BaseballException {}
  public StormyInning(String s)
    throws Foul, BaseballException {}
  // Regular methods must conform to base class:
//! void walk() throws PopFoul {} //Compile error
  // Interface CANNOT add exceptions to existing
  // methods from the base class:
//! public void event() throws RainedOut {}
  // If the method doesn't already exist in the
  // base class, the exception is OK:
  public void rainHard() throws RainedOut {}
  // You can choose to not throw any exceptions,
  // even if the base version does:
  public void event() {}
  // Overridden methods can throw inherited exceptions:
  public void atBat() throws PopFoul {}
  public static void main(String[] args) {
    try {
      StormyInning si = new StormyInning();
      si.atBat();
    } catch(PopFoul e) {
      System.out.println("Pop foul");
    } catch(RainedOut e) {
      System.out.println("Rained out");
    } catch(BaseballException e) {
      System.out.println("Generic baseball exception");
    }
    // Strike not thrown in derived version.
    try {
      // What happens if you upcast?
      Inning i = new StormyInning();
      i.atBat();
      // You must catch the exceptions from the
      // base-class version of the method:
    } catch(Strike e) {
      System.out.println("Strike");
    } catch(Foul e) {
      System.out.println("Foul");
    } catch(RainedOut e) {
      System.out.println("Rained out");
    } catch(BaseballException e) {
      System.out.println("Generic baseball exception");
    }
  }
} ///:~
```

480

在**Inning**类中，可以看到构造器和**event()**方法都声明将抛出异常，但实际上没有抛出。这种方式使你能强制用户去捕获可能在覆盖后的**event()**版本中增加的异常，所以它很合理。这对于抽象方法同样成立，比如**atBat()**。

接口**Storm**值得注意，因为它包含了一个在**Inning**中定义的方法**event()**和一个不在**Inning**中

定义的方法**rainHard()**。这两个方法都抛出新的异常**RainedOut**。如果**StormyInning**类在扩展**Inning**类的同时又实现了**Storm**接口，那么**Storm**里的**event()**方法就不能改变在**Inning**中的**event()**方法的异常接口。否则的话，在使用基类的时候就不能判断是否捕获了正确的异常，所以这也很合理。当然，如果接口里定义的方法不是来自于基类，比如**rainHard()**，那么此方法抛出什么样的异常都没有问题。

异常限制对构造器不起作用。你会发现**StormyInning**的构造器可以抛出任何异常，而不必理会基类构造器所抛出的异常。然而，因为基类构造器必须以这样或那样的方式被调用（这里默认构造器将自动被调用），派生类构造器的异常说明必须包含基类构造器的异常说明。 481

派生类构造器不能捕获基类构造器抛出的异常。

StormyInning.walk()不能通过编译的原因是因为：它抛出了异常，而**Inning.walk()**并没有声明此异常。如果编译器允许这么做的话，就可以在调用**Inning.walk()**的时候不用做异常处理了，而且当把它替换成**Inning** 的派生类的对象时，这个方法就有可能会抛出异常，于是程序就失灵了。通过强制派生类遵守基类方法的异常说明，对象的可替换性得到了保证。

覆盖后的**event()**方法表明，派生类方法可以不抛出任何异常，即使它是基类所定义的异常。同样这是因为，假使基类的方法会抛出异常，这样做也不会破坏已有的程序，所以也没有问题。类似的情况出现在**atBat()**身上，它抛出的是**PopFoul**，这个异常是继承自“会被基类的**atBat()**抛出”的**Foul**。这样，如果你写的代码是同**Inning**打交道，并且调用了它的**atBat()**的话，那么肯定能捕获**Foul**。而**PopFoul**是由**Foul**派生出来的，因此异常处理程序也能捕获**PopFoul**。

最后一个值得注意的地方是**main()**。这里可以看到，如果处理的刚好是**StormyInning**对象的话，编译器只会强制要求你捕获这个类所抛出的异常。但是如果将它向上转型成基类型，那么编译器就会（正确地）要求你捕获基类的异常。所有这些限制都是为了能产生更为强壮的异常处理代码[⊖]。

尽管在继承过程中，编译器会对异常说明做强制要求，但异常说明本身并不属于方法类型的一部分，方法类型是由方法的名字与参数的类型组成的。因此，不能基于异常说明来重载方法。此外，一个出现在基类方法的异常说明中的异常，不一定会出现在派生类方法的异常说明里。这点同继承的规则明显不同，在继承中，基类的方法必须出现在派生类里，换句话说，在继承和覆盖的过程中，某个特定方法的“异常说明的接口”不是变大了而是变小了——这恰好和类接口在继承时的情形相反。 482

练习20：(3) 修改**StormyInning.java**，加一个**UmpireArgument**异常，和一个能抛出此异常的方法。测试一下修改后的异常继承体系。

12.10 构造器

有一点很重要，即你要时刻询问自己“如果异常发生了，所有东西能被正确的清理吗？”尽管大多数情况下是非常安全的，但涉及构造器时，问题就出现了。构造器会把对象设置成安全的初始状态，但还会有别的动作，比如打开一个文件，这样的动作只有在对象使用完毕并且用户调用了特殊的清理方法之后才能得以清理。如果在构造器内抛出了异常，这些清理行为也许就不能正常工作了。这意味着在编写构造器时要格外细心。

读者也许会认为使用**finally**就可以解决问题。但问题并非如此简单，因为**finally**会每次都执

⊖ ISO C++中加上了类似的约束，要求派生类的方法所抛出的异常要与基类方法相同，或者是基类方法抛出的异常的派生类。这是C++真正能够在编译时对异常说明进行检查的唯一情况。

行清理代码。如果构造器在其执行过程中半途而废，也许该对象的某些部分还没有被成功创建，而这些部分在**finally**子句中却是要被清理的。

在下面的例子中，建立了一个**InputFile**类，它能打开一个文件并且每次读取其中的一行。这里使用了Java标准输入/输出库中的**FileReader**和**BufferedReader**类（将在第18章讨论），这些类的基本用法很简单，读者应该很容易明白：

```
//: exceptions/InputFile.java
// Paying attention to exceptions in constructors.
import java.io.*;

public class InputFile {
  private BufferedReader in;
  public InputFile(String fname) throws Exception {
    try {
      in = new BufferedReader(new FileReader(fname));
      // Other code that might throw exceptions
    } catch(FileNotFoundException e) {
      System.out.println("Could not open " + fname);
      // Wasn't open, so don't close it
      throw e;
    } catch(Exception e) {
      // All other exceptions must close it
      try {
        in.close();
      } catch(IOException e2) {
        System.out.println("in.close() unsuccessful");
      }
      throw e; // Rethrow
    } finally {
      // Don't close it here!!!
    }
  }
  public String getLine() {
    String s;
    try {
      s = in.readLine();
    } catch(IOException e) {
      throw new RuntimeException("readLine() failed");
    }
    return s;
  }
  public void dispose() {
    try {
      in.close();
      System.out.println("dispose() successful");
    } catch(IOException e2) {
      throw new RuntimeException("in.close() failed");
    }
  }
} ///:~
```

483

InputFile的构造器接受字符串作为参数，该字符串表示所要打开的文件名。在**try**块中，会使用此文件名建立了**FileReader**对象。**FileReader**对象本身用处并不大，但可以用它来建立**BufferedReader**对象。注意，使用**InputFile**的好处就能是把两步操作合而为一。

484

如果**FileReader**的构造器失败了，将抛出**FileNotFoundException**异常。对于这个异常，并不需要关闭文件，因为这个文件还没有被打开。而任何其他捕获异常的**catch**子句必须关闭文件，因为在它们捕获到异常之时，文件已经打开了（当然，如果还有其他方法能抛出**FileNotFoundException**，这个方法就显得有些投机取巧了。这时，通常必须把这些方法分别放到各自的try块里）。**close()**方法也可能会抛出异常，所以尽管它已经在另一个**Catch**子句块里了，

还是要再用一层try-catch——对Java编译器而言，这只不过是又多了一对花括号。在本地做完处理之后，异常被重新抛出，对于构造器而言这么做是很合适的，因为你总不希望去误导调用方，让他认为“这个对象已经创建完毕，可以使用了”。

在本例中，由于**finally**会在每次完成构造器之后都执行一遍，因此它实在不该是调用**close()**关闭文件的地方。我们希望文件在**InputFile**对象的整个生命周期内都处于打开状态。

getLine()方法会返回表示文件下一行内容的字符串。它调用了能抛出异常的**readLine()**，但是这个异常已经在方法内得到处理，因此**getLine()**不会抛出任何异常。在设计异常时有一个问题：应该把异常全部放在这一层处理；还是先处理一部分，然后再向上层抛出相同的（或新的）异常；又或者是不做任何处理直接向上层抛出。如果用法恰当的话，直接向上层抛出的确能简化编程。在这里，**getLine()**方法将异常转换为**RuntimeException**，表示一个编程错误。

用户在不再需要**InputFile**对象时，就必须调用**dispose()**方法，这将释放**BufferedReader**和/或**FileReader**对象所占用的系统资源（比如文件句柄），在使用完**InputFile**对象之前是不会调用它的。可能你会考虑把上述功能放到**finalize()**里面，但我在第5章讲过，你不知道**finalize()**会不会被调用（即使能确定它将被调用，也不知道在什么时候调用）。这也是Java的缺陷：除了内存的清理之外，所有的清理都不会自动发生。所以必须告诉客户端程序员，这是他们的责任。

对于在构造阶段可能会抛出异常，并且要求清理的类，最安全的使用方式是使用嵌套的**try**子句：

```
//: exceptions/Cleanup.java
// Guaranteeing proper cleanup of a resource.

public class Cleanup {
  public static void main(String[] args) {
    try {
      InputFile in = new InputFile("Cleanup.java");
      try {
        String s;
        int i = 1;
        while((s = in.getLine()) != null)
          ; // Perform line-by-line processing here...
      } catch(Exception e) {
        System.out.println("Caught Exception in main");
        e.printStackTrace(System.out);
      } finally {
        in.dispose();
      }
    } catch(Exception e) {
      System.out.println("InputFile construction failed");
    }
  }
} /* Output:
dispose() successful
*///:~
```

485

请仔细观察这里的逻辑：对**InputFile**对象的构造在其自己的**try**语句块中有效，如果构造失败，将进入外部的**catch**子句，而**dispose()**方法不会被调用。但是，如果构造成功，我们肯定想确保对象能够被清理，因此在构造之后立即创建了一个新的**try**语句块。执行清理的**finally**与内部的**try**语句块相关联。在这种方式中，**finally**子句在构造失败时是不会执行的，而在构成成功时将总是执行。

这种通用的清理惯用法在构造器不抛出任何异常时也应该运用，其基本规则是：在创建需要清理的对象之后，立即进入一个**try-finally**语句块：

```
//: exceptions/CleanupIdiom.java
// Each disposable object must be followed by a try-finally

class NeedsCleanup { // Construction can't fail
  private static long counter = 1;
  private final long id = counter++;
  public void dispose() {
    System.out.println("NeedsCleanup " + id + " disposed");
  }
}

class ConstructionException extends Exception {}

class NeedsCleanup2 extends NeedsCleanup {
  // Construction can fail:
  public NeedsCleanup2() throws ConstructionException {}
}

public class CleanupIdiom {
  public static void main(String[] args) {
    // Section 1:
    NeedsCleanup nc1 = new NeedsCleanup();
    try {
      // ...
    } finally {
      nc1.dispose();
    }

    // Section 2:
    // If construction cannot fail you can group objects:
    NeedsCleanup nc2 = new NeedsCleanup();
    NeedsCleanup nc3 = new NeedsCleanup();
    try {
      // ...
    } finally {
      nc3.dispose(); // Reverse order of construction
      nc2.dispose();
    }

    // Section 3:
    // If construction can fail you must guard each one:
    try {
      NeedsCleanup2 nc4 = new NeedsCleanup2();
      try {
        NeedsCleanup2 nc5 = new NeedsCleanup2();
        try {
          // ...
        } finally {
          nc5.dispose();
        }
      } catch(ConstructionException e) { // nc5 constructor
        System.out.println(e);
      } finally {
        nc4.dispose();
      }
    } catch(ConstructionException e) { // nc4 constructor
      System.out.println(e);
    }
  }
} /* Output:
NeedsCleanup 1 disposed
NeedsCleanup 3 disposed
NeedsCleanup 2 disposed
NeedsCleanup 5 disposed
NeedsCleanup 4 disposed
*///:~
```

486

487

在**main()**中，Section 1相当简单：遵循了在可去除对象之后紧跟**try-finally**的原则。如果对象构造不能失败，就不需要任何**catch**。在Section 2中，为了构造和清理，可以看到具有不能失败的构造器的对象可以群组在一起。

Section 3展示了如何处理那些具有可以失败的构造器，且需要清理的对象。为了正确处理这种情况，事情变得很棘手，因为对于每一个构造，都必须包含在其自己的**try-finally**语句块中，并且每一个对象构造必须都跟随一个**try-finally**语句块以确保清理。

本例中的异常处理的棘手程度，对于应该创建不能失败的构造器是一个有力的论据，尽管这么做并非总是可行。

注意，如果**dispose()**可以抛出异常，那么你可能需要额外的**try**语句块。基本上，你应该仔细考虑所有的可能性，并确保正确处理每一种情况。

练习21：(2) 试证明，派生类的构造器不能捕获它的基类构造器所抛出的异常。

练习22：(2) 创建一个名为**FailingConstructor.java**的类，它具有一个可能会在构造过程中失败并且会抛出一个异常的构造器。在**main()**中，编写能够确保不出现故障的代码。 488

练习23：(4) 在前一个练习中添加一个**dispose()**方法。修改**FailingConstructor**，使其构造器可以将那些可去除对象之一当作一个成员对象创建，然后该构造器可能会抛出一个异常，之后它将创建第二个可去除成员对象。编写能够确保不出现故障的代码，并在**main()**中验证所有可能的故障情形都被覆盖了。

练习24：(2) 在**FailingConstructor**类中添加一个**dispose()**方法，并编写代码正确使用这个类。

12.11 异常匹配

抛出异常的时候，异常处理系统会按照代码的书写顺序找出“最近”的处理程序。找到匹配的处理程序之后，它就认为异常将得到处理，然后就不再继续查找。

查找的时候并不要求抛出的异常同处理程序所声明的异常完全匹配。派生类的对象也可以匹配其基类的处理程序，就像这样：

```
//: exceptions/Human.java
// Catching exception hierarchies.

class Annoyance extends Exception {}
class Sneeze extends Annoyance {}

public class Human {
  public static void main(String[] args) {
    // Catch the exact type:
    try {
      throw new Sneeze();
    } catch(Sneeze s) {
      System.out.println("Caught Sneeze");
    } catch(Annoyance a) {
      System.out.println("Caught Annoyance");
    }
    // Catch the base type:
    try {
      throw new Sneeze();
    } catch(Annoyance a) {
      System.out.println("Caught Annoyance");
    }
  }
} /* Output:
Caught Sneeze
Caught Annoyance
*///:~
```

489

Sneeze异常会被第一个匹配的**catch**子句捕获，也就是程序里的第一个。然而如果将这个**catch**子句删掉，只留下**Annoyance**的**catch**子句，该程序仍然能运行，因为这次捕获的是**Sneeze**的基类。换句话说，**catch(Annoyance e)**会捕获**Annoyance**以及所有从它派生的异常。这一点非常有用，因为如果决定在方法里加上更多派生异常的话，只要客户程序员捕获的是基类异常，那么它们的代码就无需更改。

如果把捕获基类的catch子句放在最前面，以此想把派生类的异常全给“屏蔽”掉，就像这样：

```
try {
  throw new Sneeze();
} catch(Annoyance a) {
  // ...
} catch(Sneeze s) {
  // ...
}
```

这样编译器就会发现**Sneeze**的**catch**子句永远也得不到执行，因此它会向你报告错误。

练习25：(2) 建立一个三层的异常继承体系，然后创建基类A，它的一个方法能抛出异常体系的基类异常。让B继承A，并且覆盖这个方法，让它抛出第二层的异常。让C继承B，再次覆盖这个方法，让它抛出第三层的异常。在**main()**里创建一个C类型的对象，把它向上转型为A，然后调用这个方法。

12.12 其他可选方式

异常处理系统就像一个*活门*（trap door），使你能放弃程序的正常执行序列。当“异常情形”发生的时候，正常的执行已变得不可能或者不需要了，这时就要用到这个“活门”。异常代表了当前方法不能继续执行的情形。开发异常处理系统的原因是，如果为每个方法所有可能发生的错误都进行处理的话，任务就显得过于繁重了，程序员也不愿意这么做。结果常常是将错误忽略。应该注意到，开发异常处理的初衷是为了方便程序员处理错误。

490

异常处理的一个重要原则是“只有在你知道如何处理的情况下才捕获异常”。实际上，异常处理的一个重要目标就是把错误处理的代码同错误发生的地点相分离。这使你能在一段代码中专注于要完成的事情，至于如何处理错误，则放在另一段代码中完成。这样以来，主干代码就不会与错误处理逻辑混在一起，也更容易理解和维护。通过允许一个处理程序去处理多个出错点，异常处理还使得错误处理代码的数量趋向于减少。

“被检查的异常”使这个问题变得有些复杂，因为它们强制你在可能还没准备好处理错误的时候被迫加上**catch**子句，这就导致了*吞食则有害*（harmful if swallowed）的问题：

```
try {
  // ... to do something useful
} catch(ObligatoryException e) {} // Gulp!
```

程序员们只做最简单的事情（包括我自己，在本书第1版中也有这个问题），常常是无意中“吞食”了异常；然而一旦这么做，虽然能通过编译，但除非你记得复查并改正代码，否则异常将会丢失。异常确实发生了，但“吞食”后它却完全消失了。因为编译器强迫你立刻写代码来处理异常，所以这种看起来最简单的方法，却可能是最糟糕的做法。

当我意识到犯了这么大一个错误时，简直吓了一大跳。在本书第2版中，我在处理程序里通过打印栈轨迹的方法“修补”了这个问题（本章中的很多例子还是使用了这种方法，看起来还是比较合适的）。虽然这样可以跟踪异常的行为，但是仍旧不知道该如何处理异常。这一节，我

们来研究一下“被检查的异常”及其并发症，以及采用什么方法来解决这些问题。

这个话题看起来简单，但实际上它不仅复杂，更重要的是还非常多变。总有人会顽固地坚持自己的立场，声称正确答案（也是他们的答案）是显而易见的。我觉得之所以会有这种观点，是因为我们使用的工具已经不是ANSI标准出台前的像C那样的弱类型语言，而是像C++和Java这样的“强静态类型语言”（也就是编译时就做类型检查的语言），这是前者所无法比拟的。当刚开始这种转变的时候（就像我一样），会觉得它带来的好处是那样明显，好像类型检查总能解决所有的问题。在此，我想结合我自己的认识过程，告诉读者我是怎样从对类型检查的绝对迷信变成持怀疑态度的；当然，很多时候它还是非常有用的，但是当它挡住我们的去路并成为障碍的时候，我们就得跨过去。只是这条界限往往并不是很清晰（我最喜欢的一句格言是：所有模型都是错误的，但有些是能用的）。 491

12.12.1 历史

异常处理起源于PL/1和Mesa之类的系统中，后来又出现在CLU、Smalltalk、Modula-3、Ada、Eiffel、C++、Python、Java以及后Java语言Ruby和C#中。Java的设计和C++很相似，只是Java的设计者去掉了一些他们认为C++设计得不好的东西。

为了能向程序员提供一个他们更愿意使用的错误处理和恢复的框架，异常处理机制很晚才被加入C++标准化过程中，这是由C++的设计者Bjarne Stroustrup所倡议的。C++的异常模型主要借鉴了CLU的做法。然而，当时其他语言已经支持异常处理了：包括Ada、Smalltalk（两者都有异常处理，但是都没有异常说明），以及Modula-3（它既有异常处理也有异常说明）。

Liskov和Snyder在他们的一篇讨论该主题的独创性论文[⊖]中指出，用瞬时风格（transient fashion）报告错误的语言（如C中）有一个主要缺陷，那就是：

“……每次调用的时候都必须执行条件测试，以确定会产生何种结果。这使程序难以阅读，并且有可能降低运行效率，因此程序员们既不愿意指出，也不愿意处理异常。” 492

因此，异常处理的初衷是要消除这种限制，但是我们又从Java的“被检查的异常”中看到了这种代码。他们继续写道：

“……要求程序员把异常处理程序的代码文本附接到会引发异常的调用上，这会降低程序的可读性，使得程序的正常思路被异常处理给破坏了。”

C++中异常的设计参考了CLU方式。Stroustrup声称其目标是减少恢复错误所需的代码。我想他这话是说给那些通常情况下都不写C的错误处理的程序员们听的，因为要把那么多代码放到那么多地方实在不是什么好差事。所以他们写C程序的习惯是，忽略所有的错误，然后使用调试器来跟踪错误。这些程序员知道，使用异常就意味着他们要写一些通常不用写的、“多出来的”代码。因此，要把他们拉到“使用错误处理”的正轨上，“多出来的”代码决不能太多。我认为，评价Java的“被检查的异常”的时候，这一点是很重要的。

C++从CLU那里还带来另一种思想：异常说明。这样，就可以用编程的方式在方法的特征签名中，声明这个方法将会抛出异常。异常说明可能有两种意思。一个是“我的代码会产生这种异常，这由你来处理”。另一个是“我的代码忽略了这些异常，这由你来处理”。学习异常处理的机制和语法的时候，我们一直在关注“你来处理”部分，但这里特别值得注意的事实是，我们通常都忽略了异常说明所表达的完整含义。

C++的异常说明不属于函数的类型信息。编译时唯一要检查的是异常说明是不是前后一致；

⊖ *Exception Handling in CLU*（Barbara Liskov和Alan Snyder: CLU的异常处理），IEEE Transactions on Software Engineering, Vol. SE-5, No. 6, 1979年11月。这篇论文在网上是找不到的，只有印刷版本，所以你得去图书馆找。

比如，如果函数或方法会抛出某些异常，那么它的重载版本或者派生版本也必须抛出同样的异常。与Java不同，C++不会在编译时进行检查以确定函数或方法是不是真的抛出异常，或者异常说明是不是完整（也就是说，异常说明有没有精确描述所有可能被抛出的异常）。这样的检查只发生在运行期间。如果抛出的异常与异常说明不符，C++会调用标准类库的**unexpected()**函数。

493

值得注意的是，由于使用了模板，C++的标准类库实现里根本没有使用异常说明。在Java中，对于范型用于异常说明的方式存在着一些限制。

12.12.2 观点

首先，Java无谓地发明了“被检查的异常”（很明显是受C++异常说明的启发，以及受C++程序员们一般对此无动于衷的事实的影响）。但是，这还只是一次尝试，目前为止还没有别的语言采用这种做法。

其次，仅从示意性的例子和小程序来看，“被检查的异常”的好处很明显。但是当程序开始变大的时候，就会带来一些微妙的问题。当然，程序不是一下就变大的，这有个过程。如果把不适用于大项目的语言用于小项目，当这些项目不断膨胀时，突然有一天你会发现，原来可以管理的东西，现在已经变得无法管理了。这就是我所说的过多的类型检查，特别是“被检查的异常”所造成的问题。

看来程序的规模是个重要因素。由于很多讨论都用小程序来做演示，因此这并不足以说明问题。一名C#的设计人员发现：

“仅从小程序来看，会认为异常说明能增加开发人员的效率，并提高代码的质量；但考察大项目的时候，结论就不同了——开发效率下降了，而代码质量只有微不足道的提高，甚至毫无提高”。[㊀]

谈到未被捕获的异常的时候，CLU的设计师们认为：

494

“我们觉得强迫程序员在不知道该采取什么措施的时候提供处理程序，是不现实的。”[㊁]

在解释为什么“函数没有异常说明就表示可以抛出任何异常”的时候，Stroustrup这样认为：

“但是，这样一来几乎所有的函数都得提供异常说明了，也就都得重新编译，而且还会妨碍它同其他语言的交互。这样会迫使程序员违反异常处理机制的约束，他们会写欺骗程序来掩盖异常。这将给没有注意到这些异常的人造成一种虚假的安全感。”[㊂]

我们已经看到这种破坏异常机制的行为了——就在Java的“被检查的异常”里。

Martin Fowler (*UML Distilled*，*Refactoring*和*Analysis Patterns*的作者)给我写了下面这段话：

“……总体来说，我觉得异常很不错，但是Java的”被检查的异常“带来的麻烦比好处要多。”

我觉得Java的当务之急应该是统一其报告错误的模型，这样所有的错误都能通过异常来报告。C++不这么做的原因是它要考虑向后兼容，要照顾那些直接忽略所有错误的C代码。但是如果你一致地用异常来报告错误，那么只要愿意，随时可以抛出异常，如果不愿意，这些错误会被传播到最上层（控制台或其他容器程序）。只有当Java修改了它那类似C++的模型，使异常成为报告错误的唯一方式，那时“被检查的异常”的额外限制也许就会变得没有那么必要了。

过去，我曾坚定地认为“被检查的异常”和强静态类型检查对开发健壮的程序是非常必要的。但是，我看到的以及我使用一些动态（类型检查）语言的亲身经历[㊃]告诉我，这些好处实际

㊀ http://discuss.develop.com/archives/wa.exe?A2=ind0011A&L=DOTNET&P=R32820。

㊁ http://discuss.develop.com/archives/wa.exe?A2=ind0011A&L=DOTNET&P=R32820。

㊂ Bjarne Stroustrup, *The C++ Programming Language, 3rd edition*（C++程序设计语言，第3版），*Addison-Wesley 1997*，第376页。

㊃ 间接经验来自于与很多资深Smalltalk程序员的谈话；直接经验则得自于使用Python（www.Python.org）。

495

上是来自于：

1) 不在于编译器是否会强制程序员去处理错误，而是要有一致的、使用异常来报告错误的模型。

2) 不在于什么时候进行检查，而是一定要有类型检查。也就是说，必须强制程序使用正确的类型，至于这种强制施加于编译时还是运行时，那倒没关系。

此外，减少编译时施加的约束能显著提高程序员的编程效率。事实上，反射和泛型就是用来补偿静态类型检查所带来的过多限制，在本书很多例子中都会见到这种情形。

我已经听到有人在指责了，他们认为这种言论会令我名誉扫地，会让文明堕落，会导致更高比例的项目失败。他们的信念是应该在编译时指出所有错误，这样才能挽救项目，这种信念可以说是无比坚定的；其实更重要的是要理解编译器的能力限制。在http://MindView.net/Books/BetterJava上的补充材料中，我强调了自动构建过程和单元测试的重要性，比起把所有的东西都说成是语法错误，它们的效果可以说是事半功倍。下面这段话是至理名言：

好的程序设计语言能帮助程序员写出好程序，但无论哪种语言都避免不了程序员用它写出了坏程序。[⊖]

不管怎么说，要让Java把“被检查的异常”从语言中去除，这种可能性看来非常渺茫。对语言来说，这个变化可能太激进了点，况且Sun的支持者们也非常强大。Sun有完全向后兼容的历史和策略，实际上所有Sun的软件都能在Sun的硬件上运行，无论它们有多么古老。然而，如果发现有些“被检查的异常”挡住了路，尤其是发现你不得不去对付那些不知道该如何处理的异常，还是有些办法的。

496

12.12.3 把异常传递给控制台

对于简单的程序，比如本书中的许多例子，最简单而又不用写多少代码就能保护异常信息的方法，就是把它们从**main()**传递到控制台。例如，为了读取信息而打开一个文件（在第12章将详细介绍），必须对**FileInputStream**进行打开和关闭操作，这就可能会产生异常。对于简单的程序，可以像这样做（本书中很多地方采用了这种方法）：

```
//: exceptions/MainException.java
import java.io.*;

public class MainException {
  // Pass all exceptions to the console:
  public static void main(String[] args) throws Exception {
    // Open the file:
    FileInputStream file =
      new FileInputStream("MainException.java");
    // Use the file ...
    // Close the file:
    file.close();
  }
} ///:~
```

注意，**main()**作为一个方法也可以有异常说明，这里异常的类型是**Exception**，它也是所有“被检查的异常”的基类。通过把它传递到控制台，就不必在**main()**里写**try-catch**子句了。（不过，实际的文件输入/输出操作比这个例子要复杂得多，所以在学习第18章之前，别高兴得太早。）

12.12.4 把“被检查的异常”转换为“不检查的异常”

在编写你自己使用的简单程序时，从**main()**中抛出异常是很方便的，但这不是通用的方法。

⊖ (Kees Koster, CDL语言的设计者，引自Eiffel语言的设计者Bertrand Meyer。) http://www.elj.com/elj/v1/n1/bm/right/。

497 问题的实质是，当在一个普通方法里调用别的方法时，要考虑到“我不知道该这样处理这个异常，但是也不想把它‘吞’了，或者打印一些无用的消息”。JDK 1.4的异常链提供了一种新的思路来解决这个问题。可以直接把“被检查的异常”包装进**RuntimeException**里面，就像这样：

```
try {
  // ... to do something useful
} catch(IDontKnowWhatToDoWithThisCheckedException e) {
  throw new RuntimeException(e);
}
```

如果想把“被检查的异常”这种功能“屏蔽”掉的话，这看上去像是一个好办法。不用“吞下”异常，也不必把它放到方法的异常说明里面，而异常链还能保证你不会丢失任何原始异常的信息。

这种技巧给了你一种选择，你可以不写**try-catch**子句和/或异常说明，直接忽略异常，让它自己沿着调用栈往上“冒泡”。同时，还可以用**getCause()**捕获并处理特定的异常，就像这样：

```
//: exceptions/TurnOffChecking.java
// "Turning off" Checked exceptions.
import java.io.*;
import static net.mindview.util.Print.*;

class WrapCheckedException {
  void throwRuntimeException(int type) {
    try {
      switch(type) {
        case 0: throw new FileNotFoundException();
        case 1: throw new IOException();
        case 2: throw new RuntimeException("Where am I?");
        default: return;
      }
    } catch(Exception e) { // Adapt to unchecked:
      throw new RuntimeException(e);
    }
  }
}

class SomeOtherException extends Exception {}
```

498

```
public class TurnOffChecking {
  public static void main(String[] args) {
    WrapCheckedException wce = new WrapCheckedException();
    // You can call throwRuntimeException() without a try
    // block, and let RuntimeExceptions leave the method:
    wce.throwRuntimeException(3);
    // Or you can choose to catch exceptions:
    for(int i = 0; i < 4; i++)
      try {
        if(i < 3)
          wce.throwRuntimeException(i);
        else
          throw new SomeOtherException();
      } catch(SomeOtherException e) {
          print("SomeOtherException: " + e);
      } catch(RuntimeException re) {
        try {
          throw re.getCause();
        } catch(FileNotFoundException e) {
          print("FileNotFoundException: " + e);
        } catch(IOException e) {
          print("IOException: " + e);
        } catch(Throwable e) {
          print("Throwable: " + e);
        }
```

```
        }
    }
} /* Output:
FileNotFoundException: java.io.FileNotFoundException
IOException: java.io.IOException
Throwable: java.lang.RuntimeException: Where am I?
SomeOtherException: SomeOtherException
*///:~
```

WrapCheckedException.throwRuntimeException()的代码可以生成不同类型的异常。这些异常被捕获并包装进了**RuntimeException**对象，所以它们成了这些运行时异常的“cause”了。

在**TurnOffChecking**里，可以不用**try**块就调用**throwRuntimeException()**，因为它没有抛出“被检查的异常”。但是，当你准备好去捕获异常的时候，还是可以用**try**块来捕获任何你想捕获的异常的。应该捕获**try**块肯定会抛出的异常，这里就是**SomeOtherException**。**RuntimeException**要放到最后去捕获。然后把**getCause()**的结果（也就是被包装的那个原始异常）抛出来。这样就把原先的那个异常给提取出来了，然后就可以用它们自己的**catch**子句进行处理。 499

本书余下部分将会在合适的时候使用这种“用**RuntimeException**来包装‘被检查的异常’”的技术。另一种解决方案是创建自己的**RuntimeException**的子类。在这种方式中，不必捕获它，但是希望得到它的其他代码都可以捕获它。

练习27：(1) 修改练习3，将异常转变为**RuntimeException**。

练习28：(1) 修改练习4，使客户的异常类继承自**RuntimeException**，并展示编译器允许你省略**try**语句块。

练习29：(1) 修改**StormyInning.java**中所有的异常类型，使它们护展**RuntimeException**，并展示这里不需要任何异常说明或**try**语句块。移除“//!”注释并展示这些方法不需要说明就可以编译。

练习30：(2) 修改**Human.java**，使异常继承自**RuntimeException**修改**main()**，使其用**TurnOffChecking.java**类处理不同类型的异常。

12.13 异常使用指南

应该在下列情况下使用异常：

1) 在恰当的级别处理问题。(在知道该如何处理的情况下才捕获异常。)
2) 解决问题并且重新调用产生异常的方法。
3) 进行少许修补，然后绕过异常发生的地方继续执行。
4) 用别的数据进行计算，以代替方法预计会返回的值。
5) 把当前运行环境下能做的事情尽量做完，然后把相同的异常重抛到更高层。 500
6) 把当前运行环境下能做的事情尽量做完，然后把不同的异常抛到更高层。
7) 终止程序。
8) 进行简化。(如果你的异常模式使问题变得太复杂，那用起来会非常痛苦也很烦人。)
9) 让类库和程序更安全。(这既是在为调试做短期投资，也是在为程序的健壮性做长期投资。)

12.14 总结

异常是Java程序设计不可分割的一部分，如果不了解如何使用它们，那你只能完成很有限的工作。正因为如此，本书专门在此介绍了异常——对于许多类库（例如提到过的I/O库），如果不处理异常，你就无法使用它们。

异常处理的优点之一就是它使得你可以在某处集中精力处理你要解决的问题，而在另一处处理你编写的这段代码中产生的错误。尽管异常通常被认为是一种工具，使得你可以在运行时报告错误并从错误中恢复，但是我一直怀疑到底有多少时候“恢复”真正得以实现了，或者能够得以实现。我认为这种情况少于10%，并且即便是这10%，也只是将栈展开到某个已知的稳定状态，而并没有实际执行任何种类的恢复性行为。无论这是否正确，我一直相信“报告”功能是异常的精髓所在。Java坚定地强调将所有的错误都以异常形式报告的这一事实，正是它远远超过诸如C++这类语言的长处之一，因为在C++这类语言中，需要以大量不同的方式来报告错误，或者根本就没有提供错误报告功能。一致的错误报告系统意味着，你再也不必对所写的每一段代码，都质问自己“错误是否正在成为漏网之鱼？”（只要你没有“吞咽”异常，这是关键所在！）。

就像你将要在后续章节中看到的，通过将这个问题甩给其他代码——即使你是通过抛出**RuntimeException**来实现这一点的——你在设计和实现时，便可以专注于更加有趣和富有挑战性的问题了。

501 ~ 502

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net处购买此文档。